

Cheat Sheet: Building Supervised Learning Models

Common supervised learning models

Process Name	Brief Description	Code Syntax
One vs One classifier (using logistic regression)	Process: This method trains one classifier for each pair of classes. Key hyperparameters:- <code>`estimator`</code>: Base classifier (e.g., logistic regression) Pros: Can work well for small datasets. Cons: Computationally expensive for large datasets. Common applications: Multiclass classification problems where the number of classes is relatively small.	<pre>1. from sklearn.multiclass import OneVsOneClassifier 2. from sklearn.linear_model import LogisticRegression 3. model = OneVsOneClassifier(LogisticRegression())</pre>
One vs All classifier (using logistic regression)	Process: Trains one classifier per class, where each classifier distinguishes between one class and the rest. Key hyperparameters:- <code>`estimator`</code>: Base classifier (e.g., Logistic Regression)- <code>`multi_class`</code>: Strategy to handle multiclass classification (<code>`ovr`</code>) Pros: Simpler and more scalable than One vs One. Cons: Less accurate for highly imbalanced classes. Common applications: Common in multiclass classification problems such as image classification.	<pre>1. from sklearn.multiclass import OneVsRestClassifier 2. from sklearn.linear_model import LogisticRegression 3. model = OneVsRestClassifier(LogisticRegression())</pre>
Decision tree classifier	Process: A tree-based classifier that splits data into smaller subsets based on feature values. Key hyperparameters:- <code>`max_depth`</code>: Maximum depth of the tree Pros: Easy to interpret and visualize. Cons: Prone to overfitting if not pruned properly. Common applications: Classification tasks, such as credit risk assessment.	<pre>1. from sklearn.tree import DecisionTreeClassifier 2. model = DecisionTreeClassifier(max_depth=5)</pre>
Decision tree regressor	Process: Similar to the decision tree classifier, but used for regression tasks to predict continuous values. Key hyperparameters:- <code>`max_depth`</code>: Maximum depth	<pre>1. from sklearn.tree import DecisionTreeRegressor 2. model = DecisionTreeRegressor(max_depth=5)</pre>

		of the treePros: Easy to interpret, handles nonlinear data.Cons: Can overfit and perform poorly on noisy data.Common applications: Regression tasks, such as predicting housing prices.	
Linear SVM classifier	Process: A linear classifier that finds the optimal hyperplane separating classes with a maximum margin.Key hyperparameters:- `C`: Regularization parameter- `kernel`: Type of kernel function (`linear`, `poly`, `rbf`, etc.)- `gamma`: Kernel coefficient (only for `rbf`, `poly`, etc.)Pros: Effective for high-dimensional spaces.Cons: Not ideal for nonlinear problems without kernel tricks.Common applications: Text classification and image recognition.	<pre>1. from sklearn.svm import SVC 2. model = SVC(kernel='linear', C=1.0)</pre>	
K-nearest neighbors classifier	Process: Classifies data based on the majority class of its nearest neighbors.Key hyperparameters:- `n_neighbors`: Number of neighbors to use- `weights`: Weight function used in prediction (`uniform` or `distance`)- `algorithm`: Algorithm used to compute the nearest neighbors (`auto`, `ball_tree`, `kd_tree`, `brute`)Pros: Simple and effective for small datasets.Cons: Computationally expensive as the dataset grows.Common applications: Recommendation systems, image recognition.	<pre>1. from sklearn.neighbors import KNeighborsClassifier 2. model = KNeighborsClassifier(n_neighbors=5, weights='uniform')</pre>	
Random Forest regressor	Process: An ensemble method using multiple decision trees to improve accuracy and reduce overfitting.Key hyperparameters:- `n_estimators`: Number of trees in the forest- `max_depth`: Maximum depth of each treePros: Less prone to overfitting than individual decision trees.Cons: Model complexity increases with the number of trees.Common applications: Regression tasks such as predicting sales or stock prices.	<pre>1. from sklearn.ensemble import RandomForestRegressor 2. model = RandomForestRegressor(n_estimators=100, max_depth=5)</pre>	
XGBoost regressor	Process: A gradient boosting method that builds trees sequentially to correct errors from previous trees.Key hyperparameters:- `n_estimators`: Number of boosting rounds- `learning_rate`: Step size to improve accuracy- `max_depth`: Maximum depth of each treePros: High accuracy and works well with large datasets.Cons: Computationally	<pre>1. import xgboost as xgb 2. model = xgb.XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=5)</pre>	

	intensive, complex to tune. Common applications: Predictive modeling, especially in Kaggle competitions.	
--	---	--



Associated functions used

Method Name	Brief Description	Code Syntax
OneHotEncoder	Transforms categorical features into a one-hot encoded matrix.	<pre>1. from sklearn.preprocessing import OneHotEncoder 2. encoder = OneHotEncoder(sparse=False) 3. encoded_data = encoder.fit_transform(categorical_data)</pre>
accuracy_score	Computes the accuracy of a classifier by comparing predicted and true labels.	<pre>1. from sklearn.metrics import accuracy_score 2. accuracy = accuracy_score(y_true, y_pred)</pre>
LabelEncoder	Encodes labels (target variable) into numeric format.	<pre>1. from sklearn.preprocessing import LabelEncoder 2. encoder = LabelEncoder() 3. encoded_labels = encoder.fit_transform(target_labels)</pre>
plot_tree	Plots a decision tree model for visualization.	<pre>1. from sklearn.tree import plot_tree 2. plot_tree(model, max_depth=3, filled=True)</pre>
normalize	Scales each feature to have zero mean and unit variance (standardization).	<pre>1. from sklearn.preprocessing import normalize 2. normalized_data = normalize(data, norm='l2')</pre>
compute_sample_weight	Computes sample weights for imbalanced datasets.	<pre>1. from sklearn.utils.class_weight import compute_sample_weight 2. weights = compute_sample_weight(class_weight='balanced', y=y)</pre>
roc_auc_score	Computes the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) for binary classification models.	<pre>1. from sklearn.metrics import roc_auc_score 2. auc = roc_auc_score(y_true, y_score)</pre>



Author

[Jeff Grossman](#)

[Abhishek Gagneja](#)

Go to next item

✓ Completed

